

SentencePiece (Kudo & Richardson, Google, 2018)

- **Old way:** split into words first with a different tool per language (Moses for English, KyTea for Japanese), then run BPE
- **Problems with that:** loses spacing info, needs hand-written rules to rebuild the text, and is hard to reproduce
- **Their idea:** skip word splitting and treat the space as a normal character (), so you can always get the original text back
- Same method works for any language, with or without spaces
- Supports BPE and Unigram, and you just pick the vocab size
- Everything (vocab, rules, normalization) is saved in one model file
- English↔Japanese results: about +1.5 BLEU over the word model with a 10x smaller vocab
- Much faster on raw Japanese (5.9s vs 216s)

Here is how to train a model, encode text into tokens/IDs, and decode them back to the original string:

```
import sentencepiece as spm

# 1. Train a model directly from a raw text file.
# (No pre-tokenization or language-specific preprocessing required!)
spm.SentencePieceTrainer.train(
    input='data/botchan.txt',
    model_prefix='m',
    vocab_size=1000
)

# 2. Load the trained model.
sp = spm.SentencePieceProcessor(model_file='m.model')

# 3. Encode raw text into subword pieces (strings) or vocabulary IDs (
text = "I saw a girl with a telescope."
pieces = sp.encode(text, out_type=str)
ids = sp.encode(text, out_type=int)

print(f"Pieces: {pieces}")
# Output: ['_I', '_saw', '_a', '_girl', '_with', '_a', '_', 'te', 'l']

print(f"IDs: {ids}")
# Output: [9, 459, 11, 939, 44, 11, 4, 142, 82, 8, 28, 21, 132, 6]

# 4. Decode IDs or pieces back into the original text.
# The reconstruction is completely lossless and reversible!
print(sp.decode(ids))
# Output: "I saw a girl with a telescope."

print(sp.decode(pieces))
# Output: "I saw a girl with a telescope."
```

Old way (pre-tokenize, then BPE)

English

Raw: Hello world.

Moses: [Hello] [world] [.]

BPE: [Hello] [wor@@] [ld] [.]

Can't tell if it was "world." or "world ." when going back

Japanese

Raw: 東京に行きました

KyTea first: [東京] [に] [行き] [まし] [た]

Needs a separate tool just for Japanese

SentencePiece (space = _)

English

Raw: Hello world.

Tokens: [_He] [ll] [o] [_world] [.]

Join + replace _ with space = Hello world. (exact)

Japanese

Raw: 東京に行きました

Tokens: [_東京] [に] [行き] [ました]

Same tool, same decode step

Bottom line: One tool, any language, and you always get the original text back